

TROUBLESHOOTING

Debug your configuration

Diagnose why CLAUDE.md, settings, hooks, MCP servers, or skills aren't taking effect. Use `/context`, `/doctor`, `/hooks`, and `/mcp` to see what actually loaded.

When Claude ignores an instruction or a feature you configured doesn't appear, the cause is usually that the file didn't load, it loaded from a different location than you expected, or another file overrode it. This guide shows how to inspect what Claude Code actually loaded so you can narrow down which applies.

For installation, authentication, and connectivity problems, see [Troubleshoot installation and login](#) instead.

See what loaded into context

The `/context` command shows everything occupying the context window for the current session, broken down by category: system prompt, memory files, skills, MCP tools, and conversation messages. Run it first to confirm whether your `CLAUDE.md`, rules, or skill descriptions are present at all.

For detail on a specific category, follow up with the dedicated command:

Command	Shows
<code>/memory</code>	Which <code>CLAUDE.md</code> and rules files loaded, plus auto-memory entries
<code>/skills</code>	Available skills from project, user, and plugin sources
<code>/agents</code>	Configured subagents and their settings
<code>/hooks</code>	Active hook configurations
<code>/mcp</code>	Connected MCP servers and their status
<code>/permissions</code>	Resolved allow and deny rules currently in effect
<code>/doctor</code>	Configuration diagnostics: invalid keys, schema errors, installation health
<code>/debug [issue]</code>	Enables debug logging for the session and prompts Claude to diagnose using the log

Command	Shows
	output and settings paths
<code>/status</code>	Active settings sources, including whether managed settings are in effect

If a memory file is missing from `/memory`, check its location against [how CLAUDE.md files load](#). Subdirectory `CLAUDE.md` files load on demand when Claude reads a file in that directory with the Read tool, not at session start.

If `/memory` confirms the file loaded but Claude still isn't following a particular instruction, the issue is likely how the instruction is written rather than whether it loaded. CLAUDE.md works well for the kinds of guidance you'd give a new teammate, such as project conventions, build commands, and where files belong.

Adherence drops when an instruction is vague enough to interpret multiple ways, when two files give conflicting direction, or when the file has grown long enough that individual rules get less attention. [Write effective instructions](#) covers the specificity, size, and structure patterns that keep adherence high.

❗ CLAUDE.md and permissions solve different problems. CLAUDE.md tells Claude how your project works so it makes good decisions. [Permissions](#) and [hooks](#) enforce limits regardless of what Claude decides. Use CLAUDE.md for “we do it this way here.” Use permissions or hooks for security boundaries and anything that must never happen, where you need a guarantee instead of guidance.

Check resolved settings

Settings merge across managed, user, project, and local scopes. Managed settings always win when present. Among the rest, the closer scope overrides the broader one in the order local, then project, then user. Some settings can also be set by command-line flags or [environment variables](#), which act as another override layer. When a setting doesn't seem to apply, the value you set is usually being overridden by another scope or an environment variable.

Run `/doctor` to validate your configuration files and surface invalid keys or schema errors. When `/doctor` reports issues, press `f` to send the diagnostic report to Claude and have it walk through fixes with you.

Run `/status` to see which settings sources are active, including whether managed settings are in effect. To understand which scope wins for a given key, see [How scopes interact](#).

Check MCP servers

Run `/mcp` to see every configured server, its connection status, and whether you have approved it for the current project. A server can be defined correctly but still not provide tools for a few common reasons:

- Project-scoped servers in `.mcp.json` require a one-time approval. If the prompt was dismissed, the server stays disabled until you approve it from `/mcp`.
- A server that fails to start shows as failed in `/mcp`. Relative file paths in `command` or `args` are a frequent cause, since they resolve against the directory you launched Claude Code from rather than the location of `.mcp.json`.
- A server that shows as connected but lists zero tools has started successfully but isn't returning a tool list. Select **Reconnect** from `/mcp`. If the count stays at zero, run `claude --debug mcp` to see the server's stderr output.

For configuration locations and scope rules, see [MCP](#).

Check hooks

Run `/hooks` to list every hook registered for the current session, grouped by event. If a hook you defined doesn't appear, it isn't being read: hooks go under the `"hooks"` key in a settings file, not in a standalone file.

If the hook appears but doesn't fire, the matcher is the usual cause. The `matcher` field is a single string that uses `|` to match multiple tool names, for example `"Edit|Write"`. On Claude Code v2.1.191 or later, `,` also works as a separator, so `"Edit,Write"` is equivalent. On earlier versions a comma falls through to regex evaluation and the matcher never matches, so use `|` if you aren't on v2.1.191 yet. A misspelled tool name fails silently for the same reason. An array value is a schema error: Claude Code shows a settings error notice, `/doctor` reports the validation failure, and the hook entry is dropped so it won't appear in `/hooks`.

Edits to `settings.json` take effect in the running session after a brief file-stability delay. You don't need to restart. If `/hooks` still shows the old definition a few seconds after saving, run `/hooks` again to refresh the view.

If `/hooks` shows the hook but it still does not fire, the next step is to watch hook evaluation live. Start a session with `claude --debug hooks` and trigger the tool call. The debug log records each event, which matchers were checked, and the hook's exit code and output. See [Debug hooks](#) for the log format and [hooks troubleshooting](#) for common failure patterns.

Test against a clean configuration

Start with `claude --safe-mode`, which launches a session with all customizations disabled, including `CLAUDE.md`, skills, plugins, hooks, MCP servers, and custom commands and agents. Authentication, model selection, built-in tools, and permissions work normally. If the problem disappears in safe mode, one of those surfaces is the cause; use the targeted checks above to find which. Managed settings deployed by your organization still partially apply, so policy-configured hooks and status line run even in safe mode.

If the problem persists in safe mode, or your settings themselves are suspect, compare against a session that loads nothing from your usual setup. Point `CLAUDE_CONFIG_DIR` at an empty directory to bypass everything under `~/.claude`, and launch from a directory that has no `.claude` folder, `.mcp.json`, or `CLAUDE.md` so project configuration is also skipped.

```
cd /tmp && CLAUDE_CONFIG_DIR=/tmp/claude-clean claude
```

The clean session has no user or project settings, hooks, MCP servers, plugins, or memory.

- Managed settings still apply if your organization deploys them, since they live at a system path outside `~/.claude`
- On Linux and Windows, you'll be prompted to log in again because credentials are stored under the configuration directory
- On macOS, credentials are in the Keychain and carry over to the clean session

If the problem disappears here, the cause is somewhere in your real `~/.claude` or project `.claude` files. Reintroduce them one at a time, by copying files into the temporary directory or by launching from your project, to find which one. If it persists in the clean session, the cause is outside your user and project configuration. Run `/status` to check whether managed settings are in effect, look for environment variables that affect Claude Code, then see Troubleshooting.

Check common causes

Most configuration surprises trace back to a small set of location and syntax rules. Check these before assuming a bug:

Symptom	Cause	Fix
Hook never fires	<code>matcher</code> is a JSON array instead of a string	Use a single string with <code> </code> to match multiple tools, for example <code>"Edit Write"</code> . See matcher patterns .
Hook never fires	<code>matcher</code> uses <code>,</code> as a separator on a version before v2.1.191	Claude Code v2.1.191 or later treats <code>,</code> as a list separator like <code> </code> . Earlier versions evaluate a comma as a literal character, so <code>"Edit,Write"</code> matches nothing. Use <code> </code> instead, or upgrade Claude Code.
Hook never fires	<code>matcher</code> value is lowercase, for example <code>"bash"</code>	Matching is case-sensitive. Tool names are capitalized: <code>Bash</code> , <code>Edit</code> , <code>Write</code> , <code>Read</code> .
Hook never fires	Hooks are defined in a standalone file instead of <code>settings.json</code>	There is no standalone hooks file for project or user config. Define hooks under the <code>"hooks"</code> key in <code>settings.json</code> . Only plugins load a separate <code>hooks/hooks.json</code> . See hook configuration .
Permissions, hooks, or env set globally are ignored	Configuration was added to <code>~/.claude.json</code>	<code>~/.claude.json</code> holds app state and UI toggles. <code>permissions</code> , <code>hooks</code> , and <code>env</code> belong in <code>~/.claude/settings.json</code> . These are two different files.
A <code>settings.json</code> value seems ignored	The same key is set in <code>settings.local.json</code>	<code>settings.local.json</code> overrides <code>settings.json</code> , and both override <code>~/.claude/settings.json</code> . See settings precedence .
Skill doesn't appear in <code>/skills</code>	Skill file is at <code>.claude/skills/name.md</code> instead of in a folder	Use a folder with <code>SKILL.md</code> inside: <code>.claude/skills/name/SKILL.md</code> .
Skill appears in <code>/skills</code> but Claude never invokes it	Skill has <code>disable-model-invocation: true</code> in its frontmatter, or its description doesn't match how you phrase the request	Check the badge in <code>/skills</code> : a “user-only” label means Claude won't trigger it on its own. See skill invocation .
Subdirectory <code>CLAUDE.md</code> instructions seem ignored	Subdirectory files load on demand, not at session start	They load when Claude reads a file in that directory with the Read tool, not at launch and not when writing or creating files there. See how CLAUDE.md files load .
Subagent ignores <code>CLAUDE.md</code> instructions	The built-in Explore and Plan agents skip <code>CLAUDE.md</code> . Custom subagents load it the same way the main conversation does	For Explore or Plan, restate the instruction in your delegating prompt. For a custom subagent, put critical instructions in the agent file body, which becomes the agent's system prompt. See what loads at startup .
Cleanup logic never runs at session end	No <code>SessionEnd</code> hook configured	Add a <code>SessionEnd</code> hook in <code>settings.json</code> . See the hook events list .

Symptom	Cause	Fix
MCP servers in <code>.mcp.json</code> never load	File is under <code>.claude/</code> or uses Claude Desktop's config format	Project MCP config goes at the repository root as <code>.mcp.json</code> , not inside <code>.claude/</code> . See MCP configuration .
MCP servers added under <code>mcpServers</code> in <code>settings.json</code> never appear	<code>settings.json</code> does not read an <code>mcpServers</code> key	Define project servers in <code>.mcp.json</code> at the repository root, or run <code>claude mcp add --scope user</code> for user-scoped servers. See MCP configuration .
Project MCP server added but doesn't appear	The one-time approval prompt was dismissed	Project-scoped servers require approval. Run <code>/mcp</code> to see status and approve.
MCP server fails to start from some directories	<code>command</code> or <code>args</code> uses a relative file path	Use absolute paths for local scripts. Executables on your <code>PATH</code> like <code>npm</code> or <code>uvx</code> work as-is.
MCP server starts without expected environment variables	Variables are in <code>settings.json</code> <code>env</code> , which doesn't propagate to MCP child processes	Set per-server <code>env</code> inside <code>.mcp.json</code> instead.
<code>Bash(rm *)</code> deny rule doesn't block <code>/bin/rm</code> or <code>find -delete</code>	Prefix rules match the literal command string, not the underlying executable	Add explicit patterns for each variant, or use a PreToolUse hook or the sandbox for a hard guarantee.

Related resources

For full reference on each configuration surface, see the dedicated page:

- [.claude directory reference](#): every config file location and what reads it
- [Settings](#): precedence order and the full key list
- [Hooks reference](#): event names, payloads, and `--debug hooks` output format
- [MCP](#): server configuration, approval, and `/mcp` output
- [Troubleshoot installation and login](#): `command not found`, `PATH`, and authentication problems
- [Troubleshooting](#): performance, hangs, and search issues